L2-2.1. Splunk: Exploring SPL

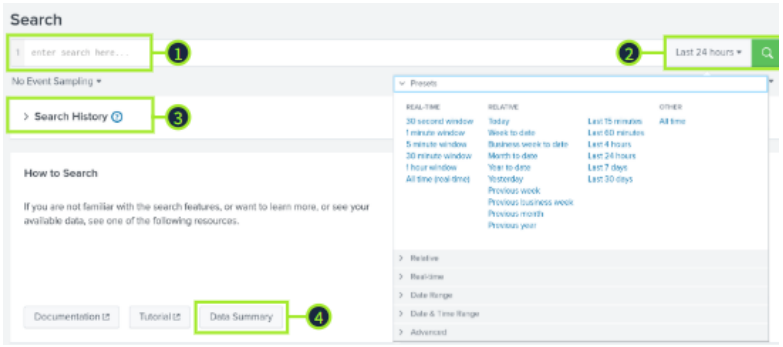
Splunk: Exploring SPL

Task 1 - Introduction

Splunk is a powerful Security Information and Event Management (SIEM) platform that allows analysts to search, visualize, and investigate log data. Its **Search Processing Language** (SPL) is the key to turning large volumes of data into practical results.

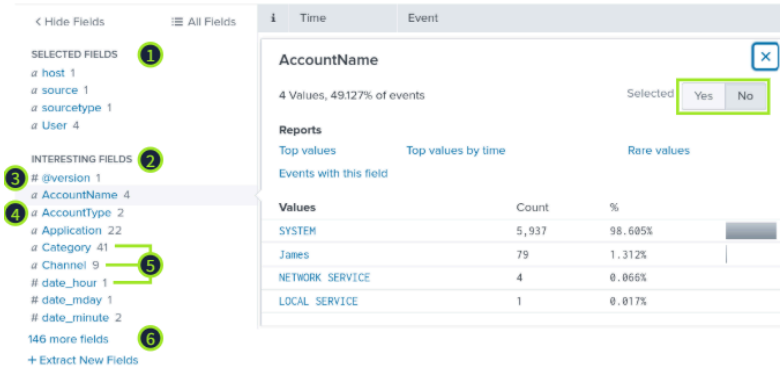
Task 2 - Search & Reporting

1. **Search Head**: Where analysts put the queies to filter or aggregate log data
2. **Time Picker**: Provides multiple options to select the timeframe of your search
3. **Search History**: Saves Splunk's search queries that have previously been used
4. **Data Summary**: Provides a summary of the hosts, sources, and sourcetypes available



Fields Sidebar

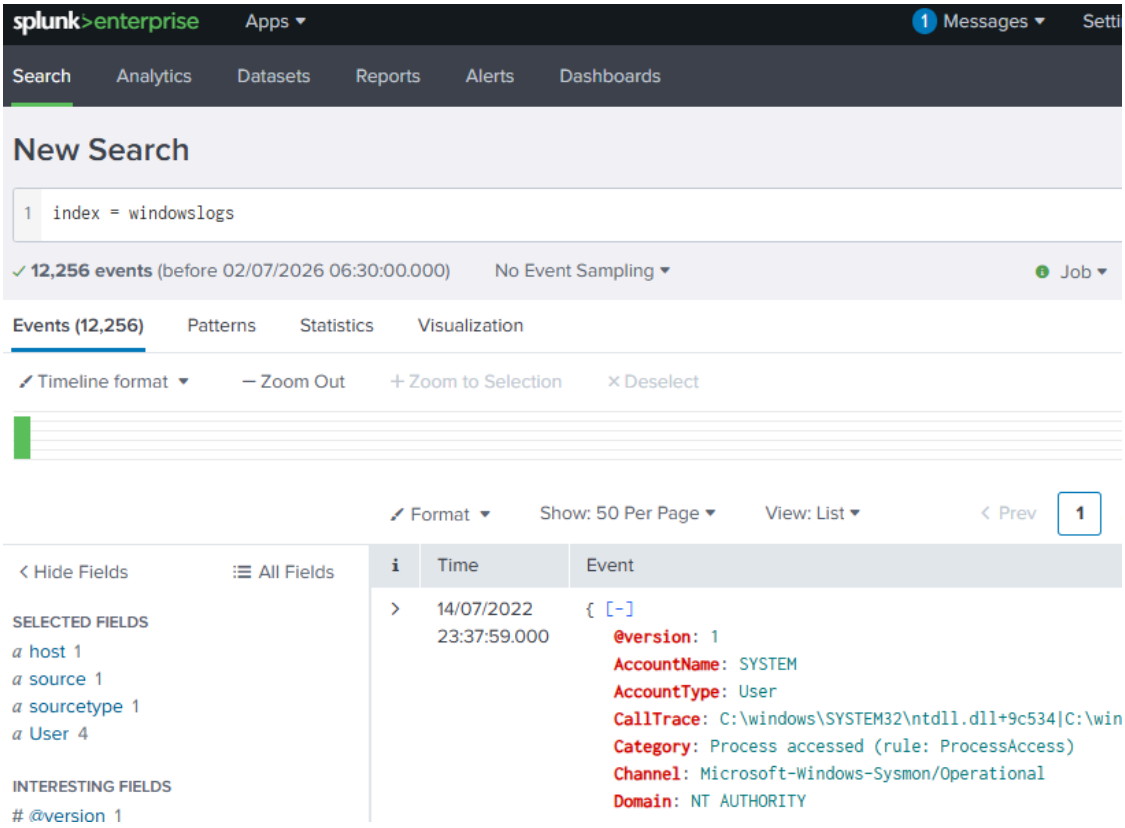
1. **Selected Fields**: The default extracted fields. You can select other fields by clicking them and toggling `Selected`
2. **Interesting Fields**: Pulls all the interesting fields it finds and displays them in the left panel to further explore
3. **Numeric Fields** `#`: This symbol represents fields that contain numerical values
4. **Alpha-numeric Fields** `a`: The alpha symbol represents fields that contain strings (text values)
5. **Count**: The number of events containing the listed field
6. **More available fields**: If more fields are available, they can be accessed and selected here



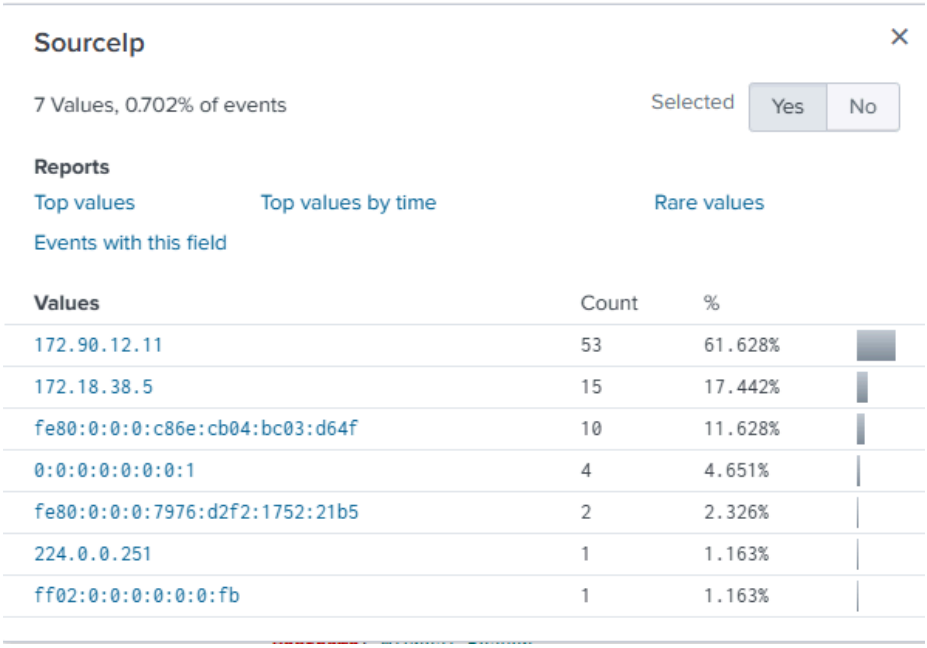
Answer the questions below

> Submit your first query for All Time: index=windowslogs.

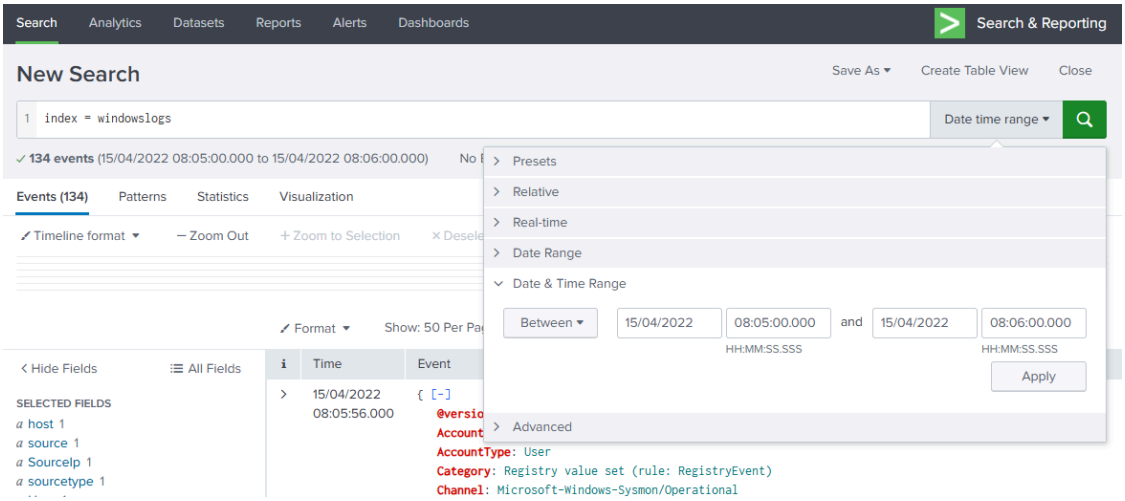
> How many total events do you see? 12256



> After you submit your first query, look in the Fields sidebar. Which SourceIP has recorded the most amount of events? 172.90.12.11



How many events appear on 04/15/2022 from 08:05 AM to 08:06 AM? 134



Task 3 - Search Operators

Search Operators (Splunk): Building blocks of search queries used to filter, exclude, and refine results based on specific criteria.

1. Relational Operators: These operators are used to compare two expressions.

Operator	Example	Explanation
Equals `=`	`UserName=Mark`	Search for all events in which the field name `UserName` is equal to `Mark`
Not Equal To `!=`	`UserName!=Mark`	Search for all events in which the field name `UserName` is not equal to `Mark`

Operator	Example	Explanation
Less Than `<`	`Age<10`	The field `Age` has a value of less than `10`
Less Than or Equal To `<=`	`Age<=10`	The field `Age` has a value of less than or equal to `10`
Greater Than `>`	`Outbound_Traffic>50`	The `Outbound_Traffic` field value is greater than `50`
Greater Than or Equal To `>=`	`Outbound_Traffic>=50`	The `Outbound_Traffic` field value is greater than or equal to `50`

2. Logical Operators: Splunk supports the following logical operators, which can be used to connect or modify conditions and operate on Boolean values (true/false).

Operator	Example	Explanation
`NOT`	`NOT UserName=*`	Returns events where `UserName` field does not exist. Don't confuse it with `!=` operator
`AND`	`UserName=David AND IPAddress=10.10.10.10`	Returns all events in which the `UserName` field is equal to `David` and the `IPAddress` field is equal to `10.10.10.10`
`OR`	`UserName=David OR UserName=John`	Returns all events in which the `UserName` field is equal to `David` or `John`
`IN`	`UserName IN(David, John)`	A more convenient alternative to the `OR` keyword, especially for long lists.

3. Wildcards and CIDR Search: Splunk supports the use of wildcards and CIDR search for IP addresses to search ffor a partial or IP subnet match. For example:

Symbol	Example	Explanation
`*`	`status=*fail*`	This will return all events that have `status` field set to `failed`, `failure`, `appfail`, etc.
`*`	`DestinationIp=172.*`	This will return all events that contain values like `DestinationIp=172.90.0.0.1` or `DestinationIp=172.18.5.22`
`N/A`	`DestinationIp=172.18.0.0/16`	This will return all events where `DestinationIp` field is within the `172.18.0.0/16` subnet

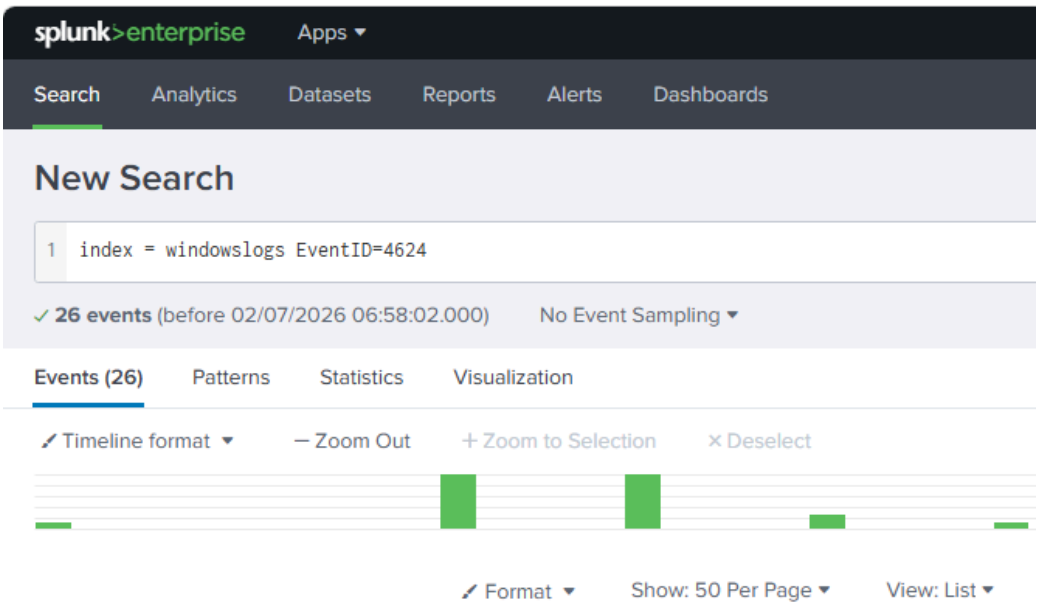
4. Order of Evaluation

Quotes: In Splunk, quotation marks `""` are used to define exact phrases or strings.

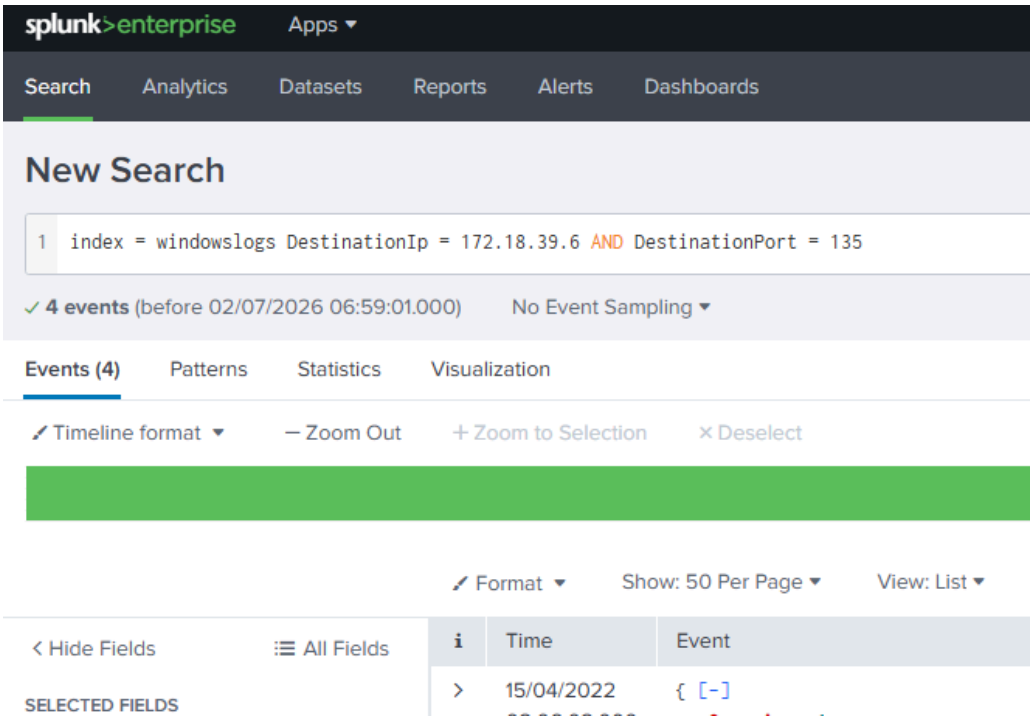
Parentheses: You can utilize parentheses in Splunk to help group conditions together and control how the search is applied. Since **OR** operator takes precedence over **AND**, parentheses can help set the correct order of conditions.

Answer the questions below

> How many events in the windowslogs index have an EventID field value equal to 4624? 26

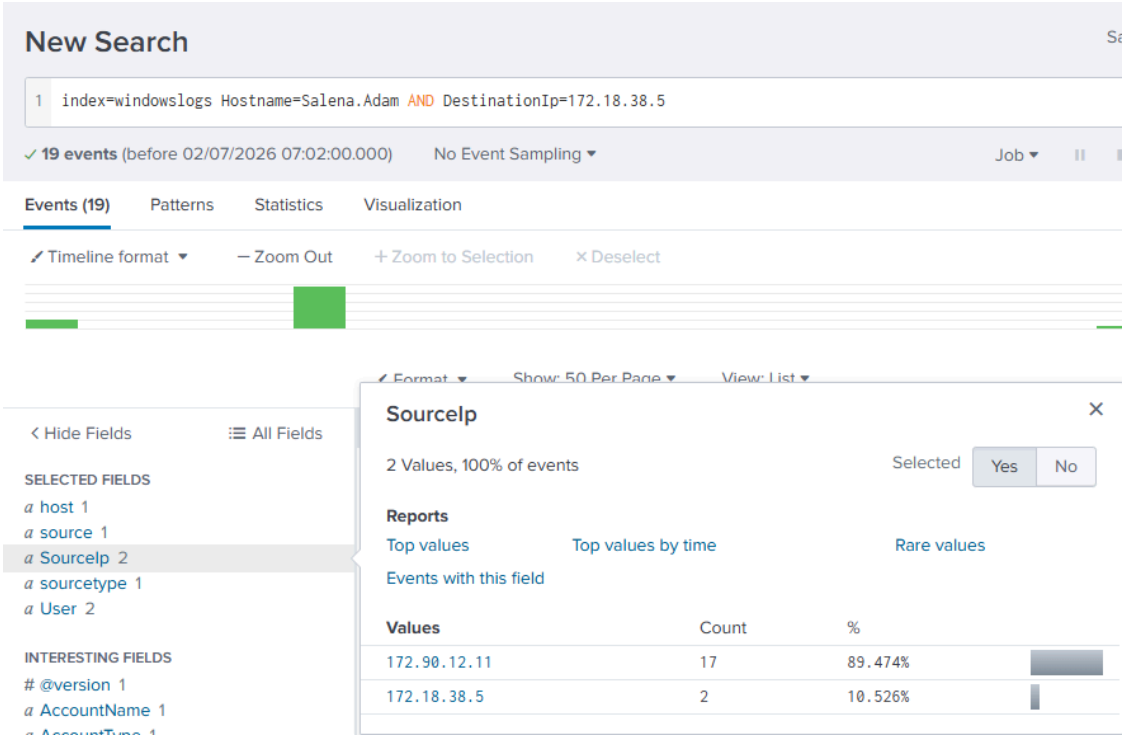


How many events are observed with the DestinationIp = 172.18.39.6 and DestinationPort = 135? 4

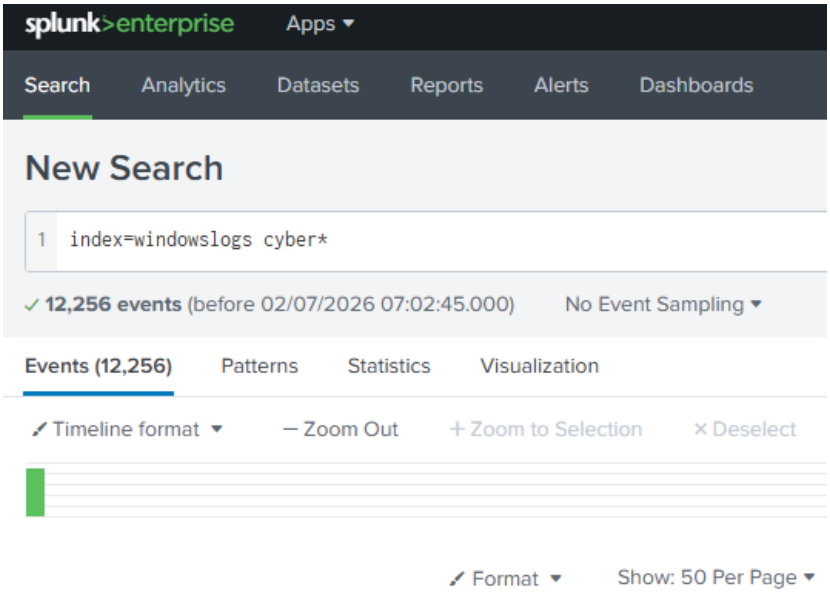


Use the query index=windowslogs Hostname=Salena.Adam DestinationIp=172.18.38.5

Which SourceIp returns the highest count? 172.90.12.11



How many events are returned when you search the term cyber*? 12256



Which operator is given the lowest priority in Splunk searches? AND

Task 4 - Filtering Results

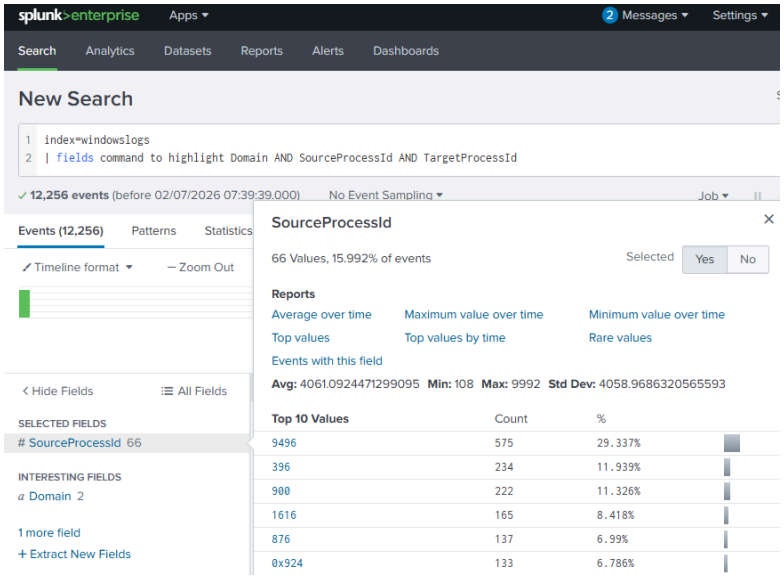
In Splunk, commands are linked together using a pipe symbol `|`. Each pipe passes the output of one command into the next, allowing you to refine your results step by step.

Useful Filtering Commands

- **fields** – Includes or excludes specific fields in search results to focus on relevant data and improve readability.
  - Example: `fields host User SourceIp`
- **dedup** – Removes duplicate values/events based on a specified field, helping reduce noise and display only unique results.
  - Example: `dedup SourceIp`
- **rename** – Changes field names to make results easier to read or simplify nested JSON/XML field structures.
  - Example: `rename User as Employee`
- **regex** – Filters events using regular expressions (PCRE) to match specific patterns within field values.
  - Example: `regex Image="\.exe$"` returns only events where the Image field ends with `.exe`

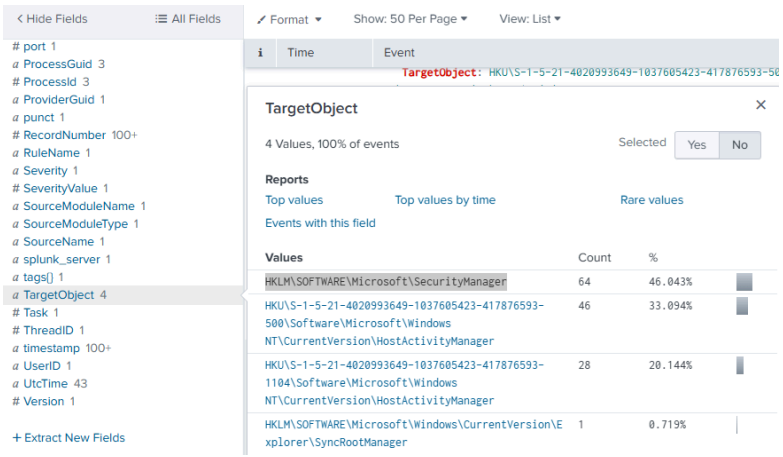
Answer the questions below

> Use the fields command to highlight Domain, SourceProcessId, and TargetProcessId.
> Which SourceProcessId has the highest value? 9496



Try out this query index=windowslogs | regex TargetObject="Manager$".

> Which TargetObject field value contains the highest number of results?
> HKLM\SOFTWARE\Microsoft\SecurityManager



Task 5- Structuring Results

Table Command (Splunk): Displays selected fields in a clean table format, making it easier to organize, analyze, and compare data such as timelines, hosts, or user activity.

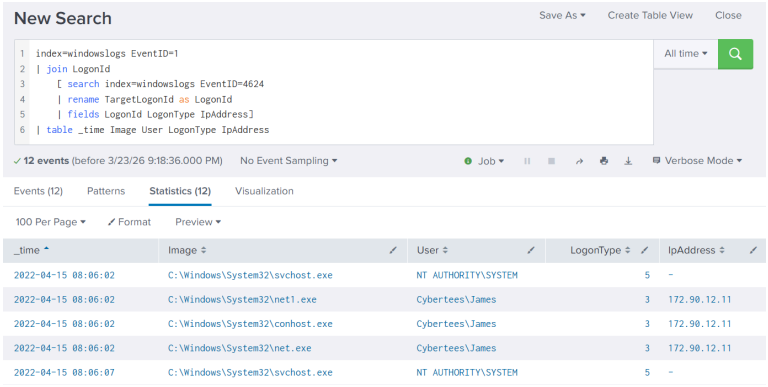
Useful Structuring Commands

Command	Example	Explanation
`head`	`index=windowslogs | head 20`	Returns the first (newest) 20 events. Useful to speed up the search if you don't need complete results
`tail`	`index=windowslogs | tail 20`	Returns the last (oldest) 10 events. Useful to speed up the search if you don't need complete results
`sort`	`index=windowslogs | sort User`	This will sort the logs in alphabetical order based on the field `User`
`reverse`	`index=windowslogs | reverse`	This command reverses the order of events (descending order)

Timelining with Table (Splunk): The `table` command can be used to build timelines by organizing event data in chronological order. It helps analysts reconstruct sequences of activity on a system (e.g., a specific host like `Salena.Adam`) while filtering out noise and focusing on relevant fields.

```
index = windowslogs Hostname = Salena.Adam
| table _time Hostname EventID Category
| reverse
```

Subsearches (Splunk): A method for correlating data from multiple sources within a single query. Subsearches (often used with `join`) allow you to match related events—such as linking Sysmon process creation logs with Windows logon events using a common field like `LogonId`—to enrich results with additional context (e.g., `LogonType`, `IpAddress`).



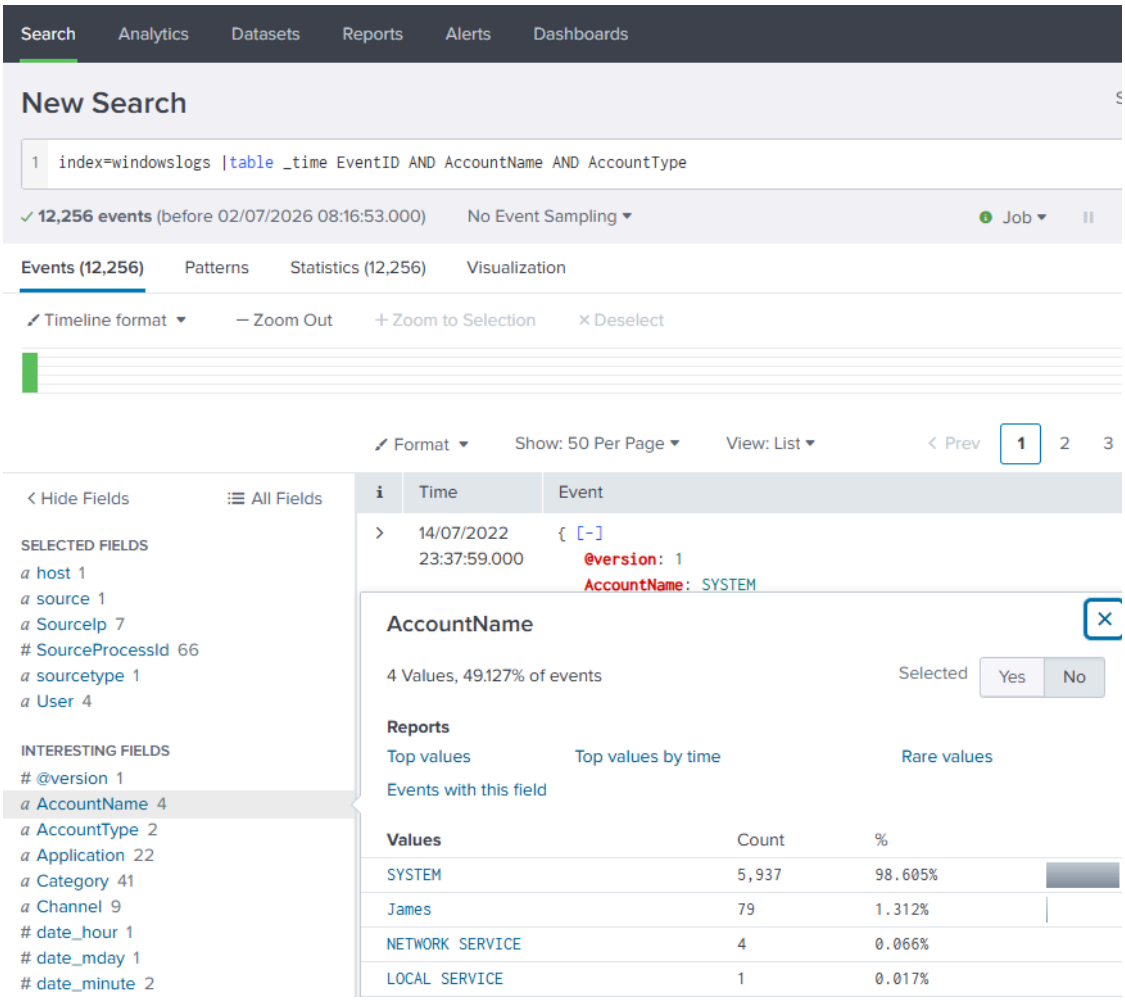

```
index=windowslogs EventID=1
| join LogonId
    [ search index=windowslogs EventID=4624
    | rename TargetLogonId as LogonId
    | fields LogonId LogonType IpAddress]
| table _time Image User LogonType IpAddress
```

`table _time` displays event timestamps in a structured table, making it easier to analyze when events occurred.

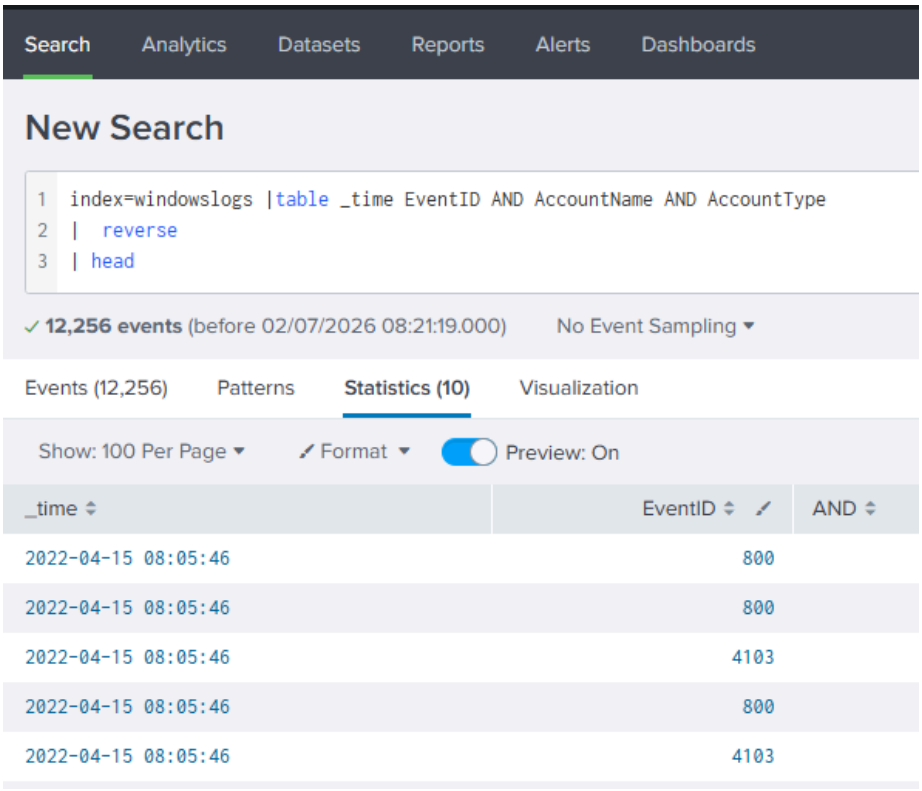
Reverse (Splunk command): Reorders search results in reverse order, typically used to display events from oldest to newest or to flip the default sorting so data can be viewed in the opposite chronological direction.

Answer the questions below

> Build a table that highlights the EventID, AccountName, and AccountType fields. Which AccountName appears first in your results? SYSTEM



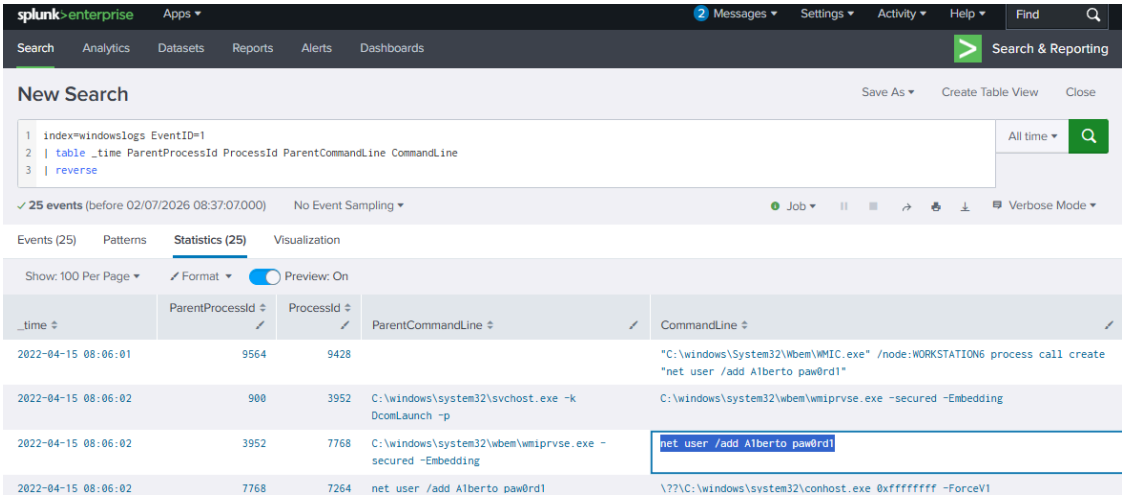
> Append the above query to include the reverse command. Which EventID appears first? 800



> Use the query below to build a timeline of events. What password was given to the user A1berto?

```
index=windowslogs EventID=1
| table _time ParentProcessId ProcessId ParentCommandLine CommandLine
| reverse
```

paw0rd1



Task 6 - Transforming Commands

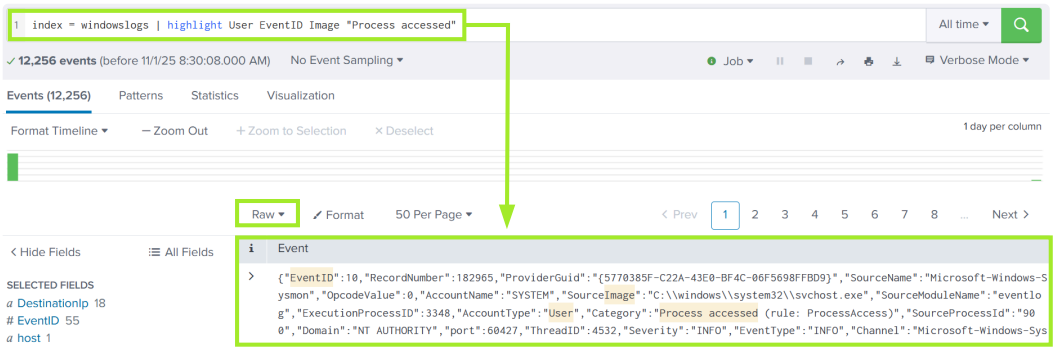
Transforming Commands (Splunk): Commands used in Splunk to convert raw event data into summaries, statistics, and visualizations. Instead of reviewing individual logs, they aggregate and analyze patterns across many events, enabling "transforming searches" that produce structured insights like counts, trends, and reports.

General Transformational Commands

Command	Example	Explanation
top	`index=windowslogs | top User limit=5`	Returns the ten most frequent values of the field specified. A numerical value can be included with `limit=` to reduce or expand the results
rare	`index=windowslogs | rare User limit=5`	Returns the ten least frequent values of the field specified. A numerical value can be included with `limit=` to reduce or expand the results

Highlight: Visually marks specific field values or keywords in raw log data to make important information easier to spot during analysis.

```
index=windowslogs | highlight User EventID Image "Process accessed"
```



Stats: A transforming command used to calculate summary statistics (e.g., count, sum, average, min, max) across fields to identify patterns or anomalies in large datasets.

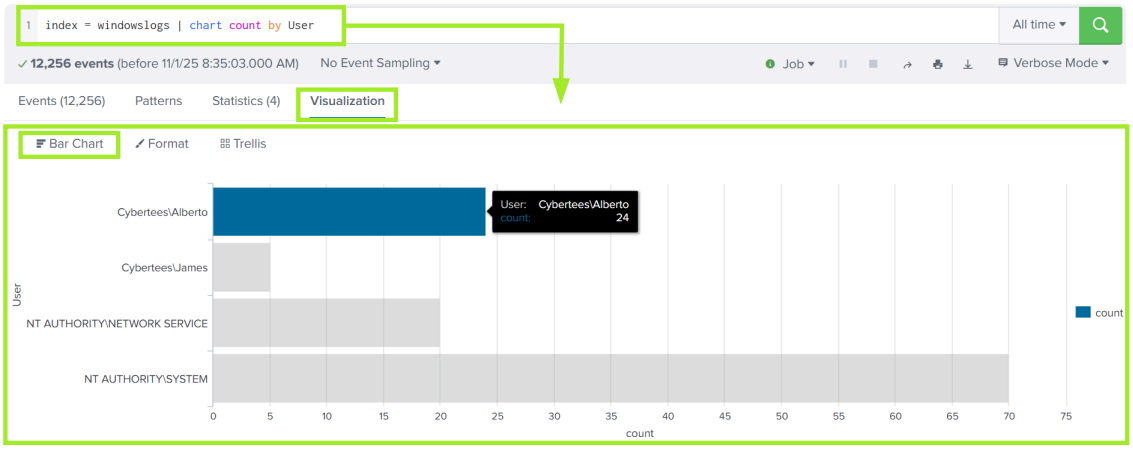
Command Function	Example	Explanation
Average	`stats avg(ProcessCount)`	Calculates the average value of the chosen field
Max	`stats max(Price)`	Returns the maximum value of the chosen field
Min	`stats min(UserAge)`	Returns the minimum value of the chosen field
Sum	`stats sum(Cost)`	Returns the sum of the chosen field values
Count	`stats count by SourceIp`	Returns the number of occurrences of the chosen field

You can try the `stats` command with the example below, which returns the total number of occurrences for each `EventID` and displays them in ascending order.

```
index=windowslogs | stats count by EventID | sort EventID
```

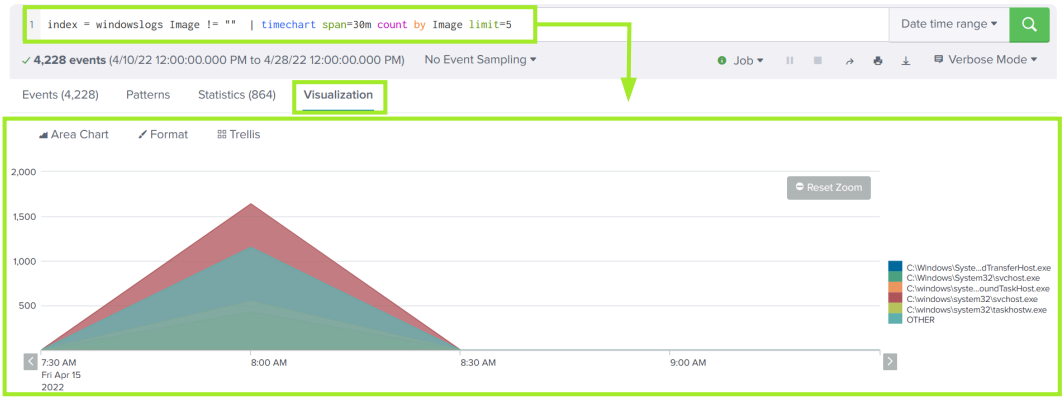
Chart: Converts search results into a tabular format designed for visualization, using aggregation functions (similar to `stats`) to compare values across fields.

```
index=windowslogs | chart count by User
```



Timechart: A time-based visualization command that shows how data changes over time, helping identify trends, spikes, and anomalies using time intervals.

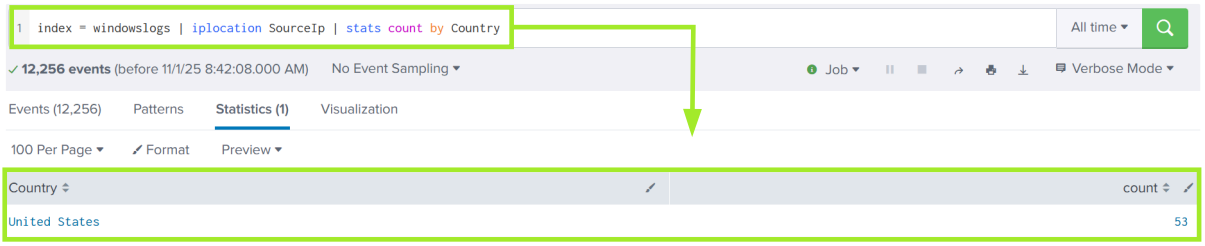
```
index=windowslogs Image!="" | timechart span=30m count by Image limit=5
```



Data Enrichment and Field Manipulation

IP Location (`iplocation`): Enriches IP addresses with geographic data such as city, region, and country using built-in geolocation lookup tables.

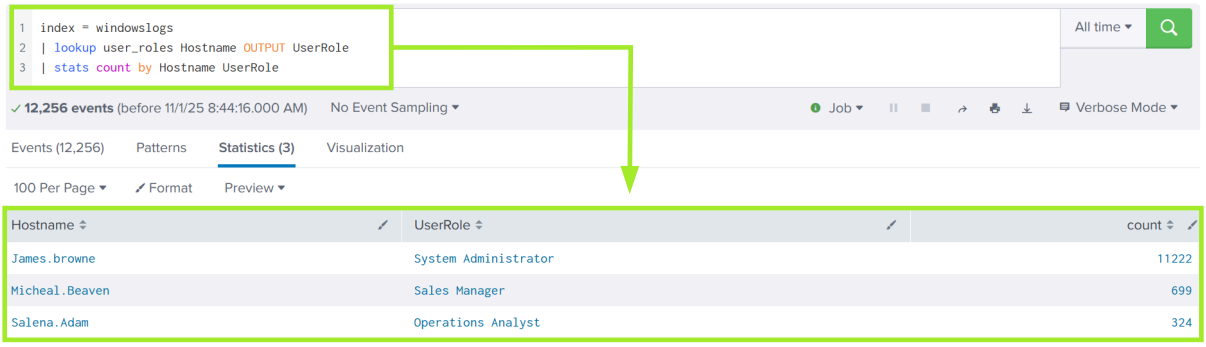
```
index=windowslogs | iplocation SourceIp | stats count by Country
```



Lookup: Enriches event data by matching fields against external sources like CSV files to add additional context (e.g., user roles or host metadata). (A CSV file (Comma-Separated Values) is a simple text file used to store data in a table format. Each line represents a row, and each value is separated by a comma.)

Example: Name,Role,Department
Alice,Analyst,Security
Bob,Engineer,IT

```
index=windowslogs
| lookup user_roles Hostname OUTPUT UserRole
| stats count by Hostname UserRole
```

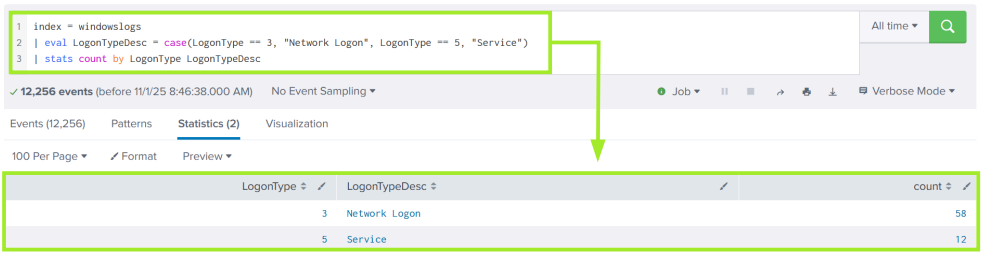


Eval: A flexible command used to create or modify fields and perform calculations, enabling data transformation and classification within searches.

```
index=windowslogs
| eval LogonTypeDesc = case(LogonType == 3, "Network Logon", LogonType == 5, "Service")
| stats count by LogonType LogonTypeDesc
```

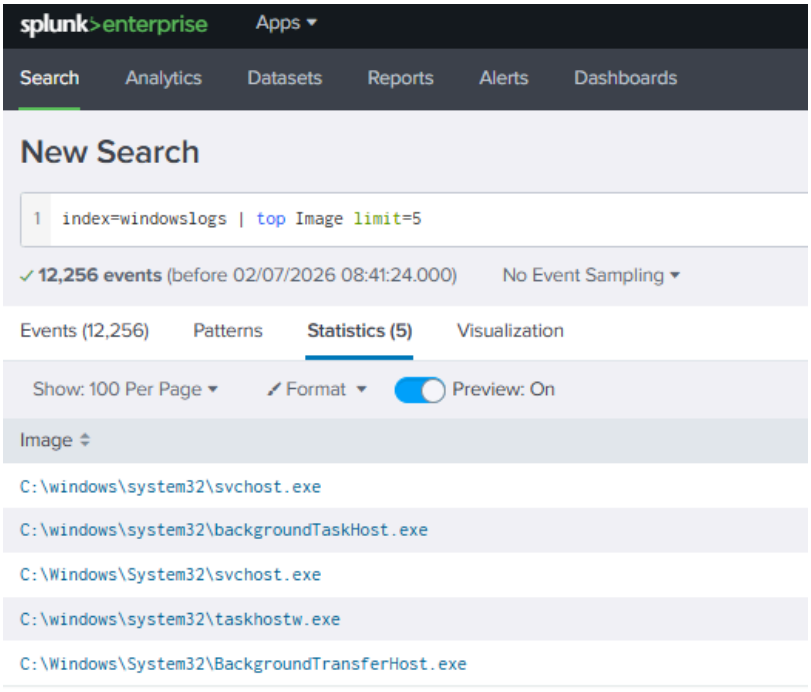
The query assigns:

- Network Logon when `LogonType` is 3
- Service when `LogonType` is 5

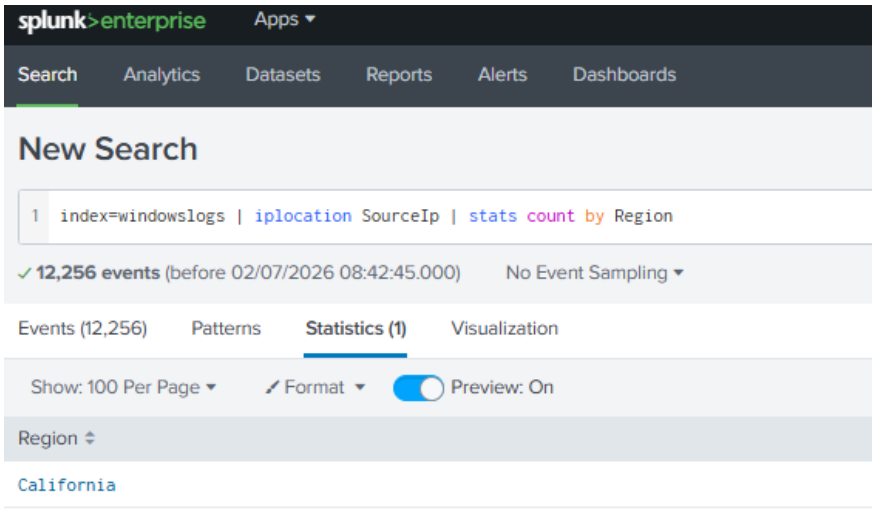


Answer the questions below

> Use the top command to query the Image field. Which Image field value has the most occurrences?
> C:\windows\system32\svchost.exe



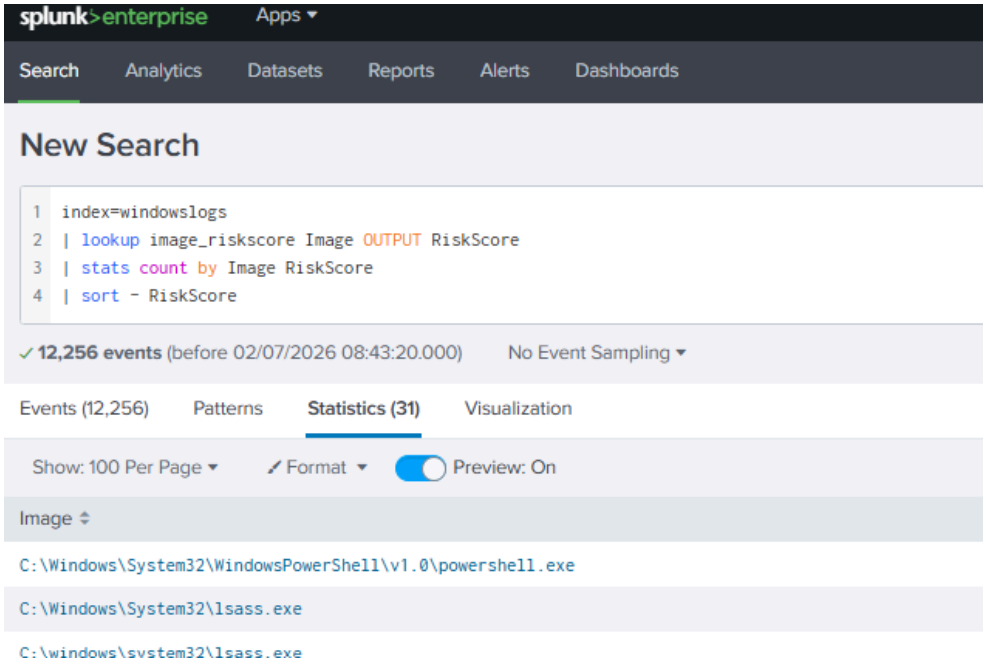
> Try out the iplocation command with the SourceIp field. Which Region do the IP addresses in your events originate from? California



> Try out this lookup query. Which Image field value has the highest RiskScore?

```
index=windowslogs
| lookup image_riskscore Image OUTPUT RiskScore
| stats count by Image RiskScore
| sort - RiskScore
```

C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

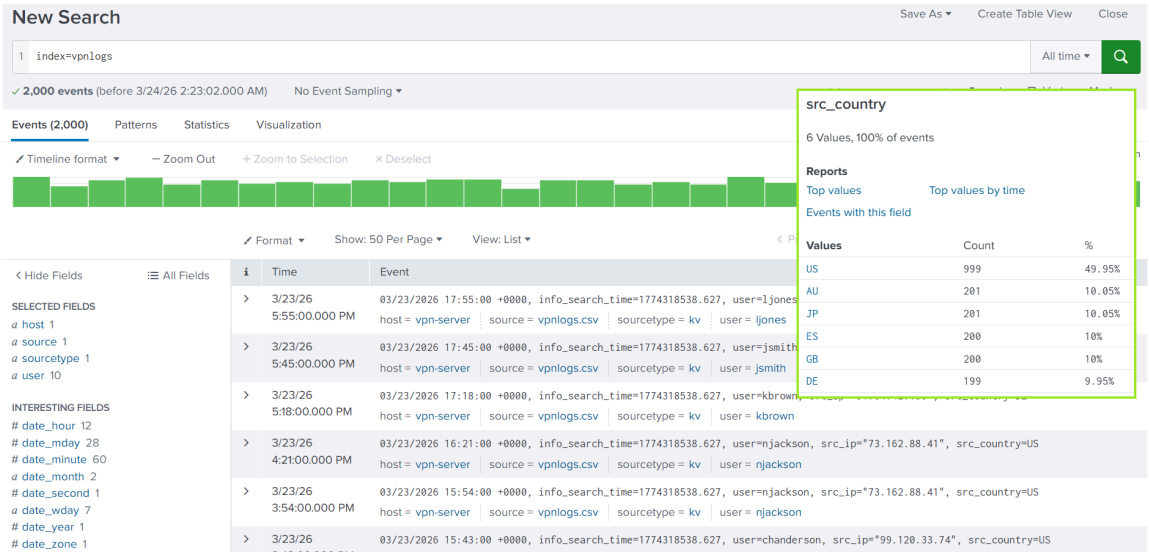


Task 7 - Anomaly Detection

Anomaly Detection (Splunk): The process of identifying unusual or suspicious events (outliers) in large datasets by comparing normal patterns of behavior against actual activity. For example, in VPN login data (with fields like time, username, source IP, and source country), anomaly detection helps identify:

- Logins from unexpected countries
- Rare or one-time locations per user
- Behavior that differs from typical user patterns

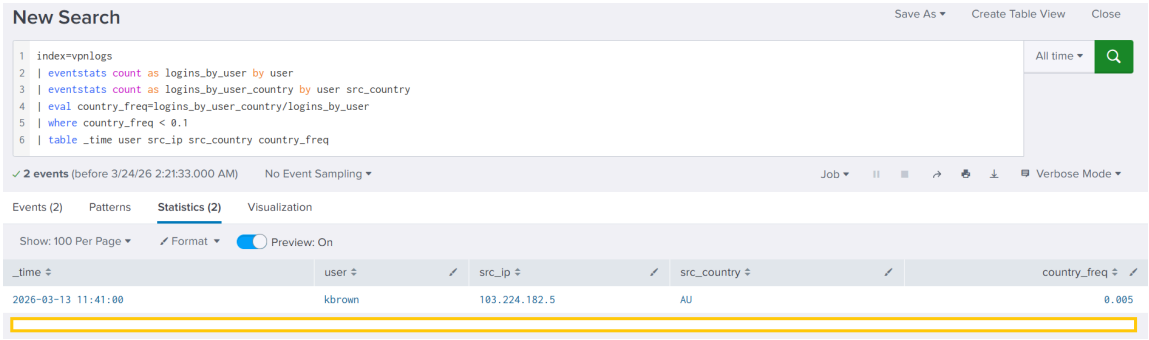
Even if basic statistics don't show obvious issues, anomaly detection focuses on **rare deviations from expected behavior**, which may indicate suspicious or malicious activity.



Detecting Outliers by Country (Splunk): A technique used to find suspicious logins by comparing each user's normal login locations against rare or unusual countries. It uses:

- `eventstats` to calculate per-user behavior while keeping raw events
- `eval` to measure how rare a country login is for each user
- `where` to filter only unusually rare user–country combinations

This helps identify users who normally log in from one country but occasionally appear from unexpected locations, which may indicate **account compromise or suspicious activity**.



Line / Command	Result
Line 2: Counts total logins by user	For kbrown user, **logins_by_user=200**
Line 3: Counts logins by user and source country	For kbrown and Austria, **logins_by_user_country=1**
Line 4: Evaluates frequency of logins per country	For kbrown and Austria, **country_freq=0.005**
Line 5: Includes only rare user-country pairs	Note: The sensitivity (0.1) is called the **threshold**
Line 6: Shows the outliers as a table	Only 2 outliers out of 2,000 login events!

Summary (Outliers by Country): The analysis identifies potentially compromised users by detecting rare login locations. In this case, users like **kbrown** and another user appear to have logged in from

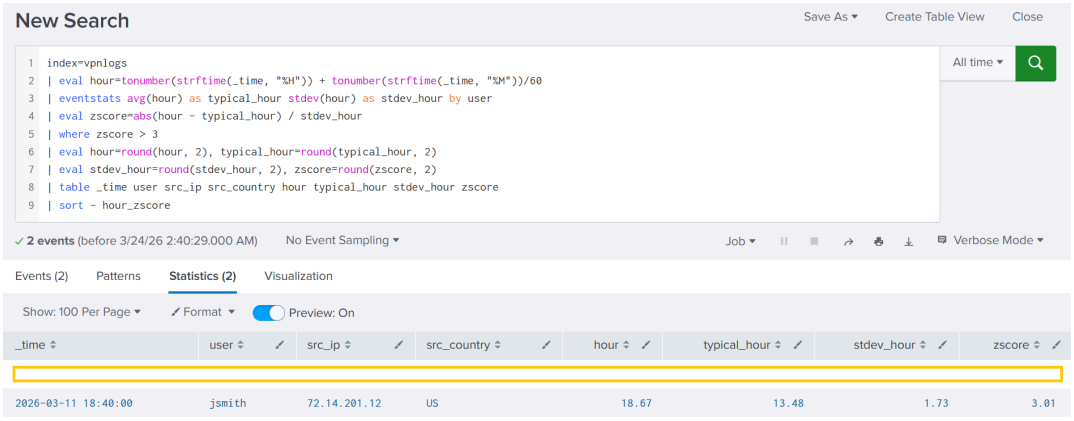
unusual countries only once, which is a strong indicator of suspicious activity such as VPN use or account compromise.

```
index=vpnlogs
| eventstats count as logins_by_user by user
| eventstats count as logins_by_user_country by user src_country
| eval country_freq=logins_by_user_country/logins_by_user
| where country_freq < 0.1
| table _time user src_ip src_country country_freq
```

Detecting Outliers by Hour (Splunk): A method for identifying unusual login times by comparing each user's observed login hour against their typical login behavior. It works by calculating:

- **Typical hour (`avg(hour)`):** the average login time per user
- **Standard deviation (`stdev(hour)`):** how consistent a user's login times are
- **Z-score:** how far a specific login deviates from the user's normal pattern

Logins with a high z-score (e.g., > 3) are considered **anomalous**, meaning they happen at unusual times compared to the user's normal behavior and may indicate suspicious activity or account misuse.




```
index=vpnlogs
| eval hour=tonumber(strftime(_time, "%H")) + tonumber(strftime(_time, "%M"))/
60
| eventstats avg(hour) as typical_hour stdev(hour) as stdev_hour by user
| eval zscore=abs(hour - typical_hour) / stdev_hour
| where zscore > 3
| eval hour=round(hour, 2), typical_hour=round(typical_hour, 2)
| eval stdev_hour=round(stdev_hour, 2), zscore=round(zscore, 2)
| table _time user src_ip src_country hour typical_hour stdev_hour zscore
| sort - hour_zscore
```

ML and Impossible Travel (Splunk): An advanced anomaly detection technique that identifies suspicious logins by analyzing whether a user appears to log in from geographically impossible

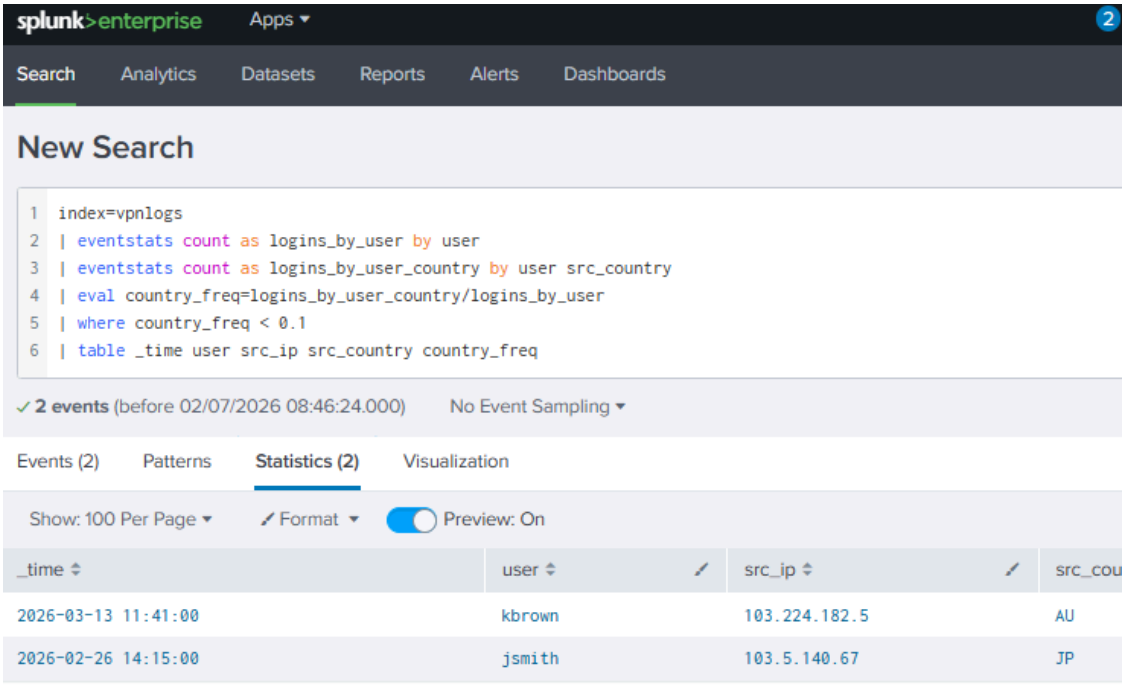
locations in a short time frame. It builds on basic SPL concepts plus:

- `iplocation` for mapping IP addresses to geographic locations
- `lookup` tables for threat intelligence enrichment
- `fit` and `apply` for applying machine learning models to detect anomalies

The goal is to detect **"impossible travel" scenarios**, where a user logs in from two distant locations in a timeframe that would be physically impossible, indicating possible account compromise or credential misuse.

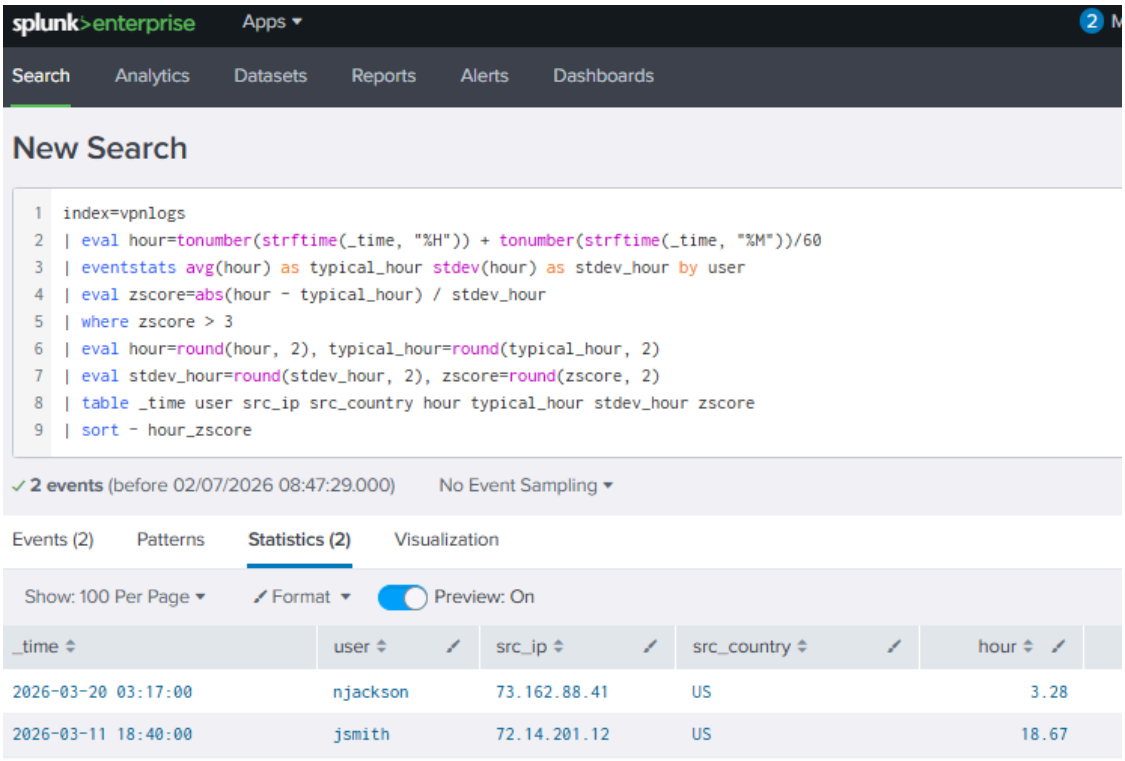
Answer the questions below

> Run the first anomaly detection query (index=vpnlogs). Which other user is marked as an outlier?
> jsmith



> Which country is anomalous for the user from Q1? JP

> Run the second anomaly detection query (index=vpnlogs). Which user suspiciously logged in at 3 AM? njackson



Task 8 - Conclusion

Learned how to use Splunk's **Search Processing Language** (SPL) to filter, structure, and transform raw log data into meaningful insights. You practiced using commands to build tables, charts, and visualizations to help uncover patterns and relationships between events.